How to change it

Most things in this kit are tied to something else. This page says what else has to change, so that the circuit, the code, the drawings and the web app stay in step.

After any change, run `node tools/check.mjs`. It tells you which files no longer agree.

Use other pins

Any free GPIO can carry I2C on this chip. Avoid GPIO2, GPIO8 and GPIO9: the chip reads them at power-on, and GPIO8 and GPIO9 are the light and the button. The battery measurement needs one of GPIO0 to GPIO4.

1. Change `SDA_PIN`, `SCL_PIN` or `BATTERY_PIN` at the top of **every** sketch in `stages/`, and in `stages/08_swinglab_bat/config.h`.
2. Change `PINS` at the top of `tools/check.mjs`.
3. Change `PINS` at the top of `diagrams/make_diagrams.py` and run it. The labels follow. The wires in the drawings do not move by themselves: for a pin on the other side of the board, move them in the script too.
4. Change the tables in `README.md` and `docs/build-on-zero-pcb.pdf`.

Give the bat another name, or use several bats

Change `DEVICE_NAME` in `config.h`. It must start with `SwingLab`, because the dashboard only lists names with that start. Give each bat its own ending, for example `SwingLab-Bat2`.

Send more or fewer readings a second

`SAMPLE_PERIOD_MS` in `config.h` is the gap between readings. 20 means 50 a second.

- The sensor takes a new reading every 10 milliseconds, so going below 10 sends the same reading twice.
- The dashboard's smoothing and its charts were made for 50 a second. Other rates work, but a shot then shows more or fewer points.
- More readings use more battery.

Change the sensor's range

A wider acceleration range would stop impacts being cut off at 8 g. But the range sets the scale, and both ends must agree on the scale.

1. In `bmi160.h`, change `ACC_RANGE_8G` and `BMI160_ACC_LSB_PER_G` together. For 16 g the register value is 0x0C and the scale is 2048 counts per g. Copy the file over its other two copies.
2. In the web app, change `ACCEL_COUNTS_PER_G` in `src/app/sensor/sensor-packet.ts` to the same number.
3. In `tools/ble_test.html`, change the 4096 in `decode` and in `FIELDS`.

If only one end changes, every acceleration on screen is wrong by a factor of two. `tools/check.mjs` compares the two scales and fails when they differ.

The rotation range is already at its widest.

Turn the axes to match the bat

See "Mount it on the bat" in the README of stage 8. Only `AXIS_SOURCE` and `AXIS_SIGN` in `config.h` change. Nothing else needs to know.

Add a value to the packet

For example the battery level, or a button.

- **Easiest: do not touch the packet.** Add a second characteristic, the way the battery level is done in the stage 8 sketch. Old versions of the web app keep working and simply ignore it.
- **If the packet itself must grow:** add the bytes at the end. The web app reads the first 14 and accepts longer packets. Then change `swing_packet.h` and its copy, `SWING_PACKET_SIZE`, the web app's `sensor-packet.ts`, and the test cases in `tools/tests/vectors.cpp` and `tools/check.mjs`. A packet must stay at 20 bytes or less to fit in one notification.
- **Never reorder the first 12 bytes.** Every version of the web app reads them by position.

Charge more slowly, or use another cell

The TP4056 module charges at 1 amp as sold. The current is set by one small resistor on the module, usually marked R3 or `Rprog` and printed `122`, which means 1.2 kilohm.

Resistor	Charging current	Suits a cell of about
1.2 kilohm, as sold	1000 mA	1000 mAh and more
2 kilohm	580 mA	600 mAh and more
3 kilohm	400 mA	400 mAh and more
5 kilohm	250 mA	250 mAh and more
10 kilohm	130 mA	130 mAh and more

- **The rule:** do not charge a cell faster than its capacity per hour, unless its own data sheet allows more. For the 1500 mAh cell that limit is 1.5 amps, so the module is fine as sold.
- **A smaller cell needs a larger resistor.** A 500 mAh cell on an unmodified module is charged at twice its limit. Replace the resistor first.
- **A larger cell** only takes longer to charge. Nothing has to change.
- **Two cells, or a 7.4 volt pack, cannot be used.** The charger and the regulator are for one cell.

Nothing in the firmware depends on the capacity.

Use other resistors for the battery measurement

The two resistors must keep the pin below 2.5 volts when the cell is at 4.2.

```
pin voltage = battery voltage x R2 / (R1 + R2)
BATTERY_DIVIDER_RATIO = (R1 + R2) / R2
```

With two equal resistors the ratio is 2. Set `BATTERY_DIVIDER_RATIO` in `config.h` and `DIVIDER_RATIO` in the stage 7 sketch. Keep the total between 100 kilohm and 1 megohm: lower wastes battery, higher makes the reading noisy.

Leave the diode out

It works, and the bat then runs until the cell is at about 3.3 volts instead of 3.5. But then **the battery must be switched off every time the ESP32's USB cable is plugged in.** If you do this, set `BATTERY_EMPTY_MILLIVOLTS` to 3350 and `BATTERY_LOW_MILLIVOLTS` to 3500.

Use another sensor

The MPU6050 is the common alternative. It needs its own driver in place of `bmi160.h`: other register numbers, other wake-up steps, and high byte first. Its scales at 8 g and 2000 degrees per second are also 4096 and 16.4, so the packet and the web app stay the same. Stage 2 tells you when the board you have is an MPU6050.

Use another ESP32 board

The firmware runs on any ESP32-C3 board. The pin numbers stay the same, but they sit in other places, so the zero PCB layout and the drawings do not apply. On a board with no light on a pin, set `LED_PIN` to -1.

Change a drawing

Every drawing is made by `diagrams/make_diagrams.py`. Change it and run:

```
python3 diagrams/make_diagrams.py
```

The zero PCB layout is data at the top of the layout section: which hole each pin sits in, and which holes are joined by a wire. The script refuses to draw a layout that does not match the circuit, or in which a wire runs through the pad of another connection.

Check your change without hardware

```
node tools/check.mjs
tools/sim/run.sh stages/08_swinglab_bat "connect@3000,swing@5000:400:1200:1,end@8000"
```

The second line runs one sketch on the computer and prints what it would print. The scenario words are listed at the top of `tools/sim/sim_main.cpp`.